Workshop Registration System

Final project exercise – student guide

1. Overview

In this exercise you build the core of a system that manages **workshop registrations**. The system keeps track of instructors, the workshops they teach, the students, and the teams that students form to take part in a workshop.

We give you the **API** (a set of Java interfaces) that describes what the system looks like and what it can do. Your job is to write the **implementation** of these interfaces. We also give you a **tester application** that you run against your implementation, so you can check on your own that it behaves as expected.

1.1 What you receive from us

- **Two jars** that contain only interfaces (no implementation):
  - `model` (model-1.0.jar) – the model objects (Person, Instructor, Student, Workshop, Team).
  - `logic` (logic-1.0.jar) – the `WorkshopSystem` interface, the entry point to all system operations.
- **The tester application** (`workshop-tester.jar`) – a black-box program that finds your implementation, runs a series of flows against it and prints a report of what was OK and what went wrong.
- **This document**, which describes the system, the rules your implementation must follow and how to use the tester.

1.2 What you need to do

1. Read this document and the interfaces in the `model` and `logic` modules.
2. Write classes that implement all the model interfaces.
3. Write a class that implements `WorkshopSystem`. It must be `public` and have a `public` constructor with no arguments.
4. Package your implementation (see section 6.1) and run the tester against it.
5. Fix problems until the tester reports **RESULT: ALL OK**.

You are free to design the internals of your implementation as you wish (data structures, helper classes, extra packages). The only thing that matters is that the public behaviour follows the API and the rules in this document. Do not change the given interfaces.

2. The System

2.1 Concept

The system is a small, in-memory registry. Everything is created through the single `WorkshopSystem` object and can later be queried through it. There is no database and no user interface – the system is used purely through Java calls.

The main ideas are:

- An **instructor** teaches one or more **workshops**.
- A **student** registers for a workshop by joining a **team** that is created for that workshop.
- A student belongs to at most one team. A student who is not in any team is called a **free student**.
- Because a team belongs to a workshop and a workshop belongs to an instructor, the system can tell, for every student, who the student's instructor is, and who the student's team mates are.

2.2 Model objects

All model interfaces are in the package `patmal.workshops.model.api`.

Person

The common base of instructors and students. Describes a human being known to the system.

Method	Meaning
`String getId()`	Unique identifier, supplied by the caller when the person is created (e.g. `s1`, `i2`).
`String getName()`	Full name.
`String getEmail()`	E-mail address.

Instructor (extends Person)

A person who teaches workshops.

Method	Meaning
`List<Workshop> getWorkshops()`	All the workshops this instructor teaches. An instructor may teach several workshops, or none yet.

Student (extends Person)

A person who takes part in a workshop as a member of a team. Most of the information about a student is **derived** from the team the student belongs to.

Method	Meaning
`String getTeamName()`	The name of the student's team, or `null` for a free student.
`Instructor getInstructor()`	The instructor of the workshop of the student's team, or `null` for a free student.
`List<Student> getTeamMates()`	The other members of the student's team (the student himself/herself is **not** included). Empty list when the student has no team mates, or no team.

Workshop

A course that is given by an instructor and that teams register to.

Method	Meaning
`Integer getId()`	Identifier assigned **by the system** when the workshop is created: a running number starting from 1.
`String getName()`	Workshop name.
`String getDescription()`	Short description.
`Instructor getInstructor()`	The instructor who teaches it. `null` until the workshop is associated with an instructor.

Team

A group of students that take a specific workshop together.

Method	Meaning
`Integer getId()`	Identifier assigned **by the system** when the team is created: a running number starting from 1.
`String getName()`	Team name.
`List<Student> getStudents()`	The members of the team.
`Workshop getWorkshop()`	The workshop the team takes part in.

2.3 Relations between the objects

```
Instructor 1 ---- * Workshop 1 ---- * Team 1 ---- 1..* Student

  instructor.getWorkshops()      workshop.getInstructor()
  team.getWorkshop()             team.getStudents()
  student.getTeamName() / getTeamMates() / getInstructor()   (derived through the
team)
```

The relations are **two-way and must stay consistent**: if a workshop reports instructor `i1`, then `i1.getWorkshops()` must contain that workshop; if a team contains student `s2`, then `s2.getTeamName()` must return the name of that team.

2.4 The WorkshopSystem interface

Located in `patmal.workshops.logic.api.WorkshopSystem`. It is the only entry point: the tester creates one instance and performs every operation through it.

Creating things

Method	What it does
`Instructor createInstructor(String id, String name, String email)`	Registers a new instructor with the given id and returns it.
`Student createStudent(String id, String name, String email)`	Registers a new student with the given id and returns him/her. The student is free (no team).
`Workshop createWorkshop(String name, String description)`	Creates a workshop and assigns it the next workshop id (1, 2, 3, ...). It has no instructor yet.
`void associateWorkshopWithInstructor(Instructor, Workshop)`	Makes the instructor the teacher of the workshop. Both sides of the relation must be updated.
`Team createTeam(String name, Workshop workshop, List<String> members)`	`members` are **student ids**. Creates a team in that workshop, assigns it the next team id (1, 2, 3, ...), and puts all the listed students in it.

Querying

Method	What it returns
`List<Instructor> getInstructors()`	All instructors.
`Instructor getInstructor(String id)`	The instructor with that id.
`List<Student> getStudents()`	All students.
`Student getStudent(String id)`	The student with that id.
`List<Workshop> getWorkshops()`	All workshops.
`Workshop getWorkshop(Integer id)`	The workshop with that id.
`List<Team> getTeams()`	All teams.

Method	What it returns
`Team getTeam(Integer id)`	The team with that id.
`List<Student> getFreeStudents()`	All students that are not in any team.
`List<Student> getStudentsOfInstructor(String instructorId)`	All students that are in a team of any workshop taught by that instructor (an instructor with several workshops gets the students of all of them).
`List<Student> getStudentsOfWorkshop(Integer workshopId)`	All students that are in a team of that workshop.

3. Rules your implementation must follow

The Java signatures do not say everything. The following rules are part of the exercise, and the tester checks them.

3.1 Identifiers

- Instructor and student ids are given by the caller and must be **unique** within their kind.
- Workshop and team ids are assigned by the system, as **sequential numbers starting from 1**, in creation order (each kind has its own counter).
- An operation that **fails must not consume an id**. If a `createTeam` call is rejected, the next successful team still gets the next number in the sequence.

3.2 Invalid operations

The following operations must be **rejected by throwing an exception** (any runtime exception is accepted; `IllegalArgumentException` is the natural choice):

- Creating a student or an instructor with an id that already exists.
- Creating a team that lists a student id that does not exist.
- Creating a team that lists a student who is already in another team.
- Creating a team that lists the same student twice.

A rejected operation must leave the system **exactly as it was**. For example, if `createTeam` is called with `["s7", "s99"]` and `s99` does not exist, then `s7` must still be a free student afterwards (do not half-apply the operation).

3.3 Unknown keys

When a lookup is made with an id that does not exist, the API does not dictate the behaviour, so both of these are accepted: return `null` (for `getX(id)`) or an empty list (for `getStudentsOfX(id)`), or throw an exception. Be consistent and document your choice in the code.

3.4 Derived data and returned lists

- `getTeamName()` and `getInstructor()` of a free student return `null`; `getTeamMates()` returns an empty list.
- `getTeamMates()` never contains the student himself/herself.
- Lists never contain the same object twice, and are never `null`.
- The order of elements inside returned lists is not important.

4. Flows your system must support

The tester runs the following flows. Every flow starts from a fresh instance of your `WorkshopSystem`, which is populated with the same reference data (section 5).

#	Flow	What is verified
1	Registration	All instructors, students, workshops and teams that were created can be listed and fetched by id, with the correct name, e-mail and description. Workshop and team ids are 1, 2, 3, ...
2	Workshops and instructors	Each workshop knows its instructor, and each instructor lists the workshops he or she teaches (including an instructor with more than one workshop).
3	Team composition	Team members and team workshop. For every student: team name, team mates (without self) and instructor. Free students return `null` / empty values.
4	Free students	`getFreeStudents()` returns exactly the students that are not in a team.
5	Students of an instructor	An instructor with several workshops, an instructor with one workshop, and an instructor that has no students.
6	Students of a workshop	Per workshop, including a workshop that has no students yet.
7	Unknown keys	Lookups of ids that do not exist behave as described in 3.3.
8	Invalid operations	The operations of 3.2 are rejected, and the system (data and id counters) is unchanged afterwards.
9	The system evolves	After the initial setup: a team is created for the workshop that had no students; a new student is added; a new workshop is created and associated with an instructor that already teaches one; the free student joins a team in the new workshop. All queries must reflect every step.

5. The tester's reference data

So that you can reproduce and debug a failing flow yourself, this is the data the tester creates before each flow:

Instructor	Name	Workshops taught	Students (via teams)
`i1`	Dana Levi	1 Java Workshop, 2 Web Development	`s1`, `s2`, `s3`
`i2`	Yossi Mizrahi	3 Databases	`s4`, `s5`, `s6`
`i3`	Noa Katz	4 Mobile Apps	none

Team id	Team name	Workshop	Members
1	Alpha	1 Java Workshop	`s1` (a team of one)
2	Beta	2 Web Development	`s2`, `s3`
3	Gamma	3 Databases	`s4`, `s5`, `s6`

In total there are 3 instructors, 4 workshops, 8 students (`s1` … `s8`) and 3 teams. Students `s7` and `s8` are **free**, and workshop 4 (and therefore instructor `i3`) has no students yet. The objects are created in exactly the order listed above.

6. Working on the exercise

6.1 Where to write your code

Write your implementation in your own classes. With this prefix package: patmal.workshops, not by editing the given interfaces. Build it so that it produces a jar.

Your implementation needs to include:

- You should implement this exercise in plain vanilla Java (JDK 25). **No** maven projects. No frameworks (spring\boot). – like you worked in the java course.
- One class implementing each model interface (Instructor, Student, Workshop, Team).
- One `public` class implementing `WorkshopSystem` with a `public` no-argument constructor. The tester finds this class automatically – there must be exactly one such class on the classpath.

6.2 Suggested way to proceed

1. Aim to split the implementation to two (intellij) modules: one for the model objects and one for the WorkshopSystem. This structure will be similar to our future evolvement of the project.
2. Start with the model classes (they are simple holders).
3. Implement the creation methods and the simple getters of `WorkshopSystem`.
4. Implement `associateWorkshopWithInstructor` and `createTeam`, keeping both sides of every relation up to date.
5. Implement the derived queries (free students, students of instructor / workshop).
6. Add the validation of section 3.2.
7. Run the tester after every step and read the failures – they tell you exactly what was expected.

6.3 Tips

- Keep data in maps keyed by id.
- Validate **everything** about a request (all member ids) before changing anything; this makes rejected operations harmless.
- Return copies of your internal lists so callers cannot corrupt your state.
- Write your own small tests as well – the tester is a safety net, not a debugger.

7. Using the tester application

7.1 What it is

The tester is a console program, `patmal.workshops.main.Main`, delivered in the `workshop-tester.jar`. It knows **only the API**. It searches the classpath for your `WorkshopSystem` implementation, creates a fresh instance for every flow, performs the operations of section 4 and compares the results with the expected ones. It never needs your source code.

7.2 What you need on the classpath

Jar / folder	Purpose
`workshop-tester.jar` (the tester)	The tester program itself.
`model` and `logic` API jars	The interfaces the tester and your code share.
Your jar(s)	Your implementation of the model and of `WorkshopSystem`.

7.3 Running from the command line

On Windows, separate the classpath entries with a semicolon (`;`); on Linux / macOS use a colon (`:`).

```
java -cp "workshop-tester.jar;logic-1.0.jar;model-1.0.jar;your-implementation.jar"
patmal.workshops.main.Main
```

7.4 Reading the report

The tester prints, for each flow, one line per check:

```
=== 3. Team composition and derived student data ===
  [PASS] team Alpha members
  [FAIL] s4 team mates (self excluded)
         expected <[s5, s6]> but was <[s4, s5, s6]>
```

At the end there is a summary per flow and a list of everything that went wrong:

```
================ SUMMARY ================
  OK     1. Instructors, students, workshops and teams are registered (23/23 checks
passed)
  FAILED 3. Team composition and derived student data (19/20 checks passed)
  ...
Scenarios: 8 passed, 1 failed
Checks:    104 passed, 1 failed

What went wrong:
  - [3. Team composition ...] s4 team mates (self excluded) -> expected <[s5, s6]>
but was <[s4, s5, s6]>
RESULT: FAILURES FOUND
```

Your goal is the line **RESULT: ALL OK**.

Exit codes

Code	Meaning
0	All checks passed.
1	At least one check failed.
2	The tester could not find (or load) exactly one implementation of `WorkshopSystem`.

7.6 Troubleshooting

Symptom	Likely cause
"No implementation of WorkshopSystem ... was found on the classpath"	Your jar / classes folder is not on the classpath, or the class is not `public`, is abstract, or has no public no-argument constructor.
"More than one implementation ... found"	Several implementations are on the classpath. Remove the extra one or use `-Dworkshop.system.class=...`.
The only failure is "set up the reference model"	One of the creation calls (`createInstructor`, `createWorkshop`, `associateWorkshopWithInstructor`, `createStudent`, `createTeam`) threw an exception. The message shows the exception and where it came from. Every flow depends on this step, so fix it first.
"unexpected NullPointerException"	A getter returned `null` where an object was expected (e.g. a lookup by id that finds nothing, or a relation that was not updated).
"expected an exception but none was thrown"	An invalid operation of section 3.2 was accepted.
Ids are off by one	A rejected operation consumed an id (see 3.1), or the counters do not start from 1.

8. What to submit?

A text file holding a (full) reference to your github repo.

Other than the code, the github repo should have a folder called `artifacts` that will hold your jar(s) implementation. These will be downloaded and tested with the tester you got before hand.

The process is automatic. Make sure:

1. File holds only a reference to your github repository. e.g., https://github.com/<user name>/<repo name>
2. The repository should hold this tag on the commit that it is considered to be your submitted implementation. The tag name is: **EX1-Vanilla** (this exact name, case sensitive).
3. Repository is not private
4. It holds the folder `artifacts` in its root